VibePop 产品优化 PRD

VibePop 产品优化 PRD

> 文档类型：产品需求文档 (PRD)创建日期：2026-04-14状态：持续更新中

优化项 1：Feed 手势交互机制

1.1 基本信息

字段	内容
优化名称	Feed 手势交互机制
优先级	P0 - MVP 核心体验
状态	待评审
提出日期	2026-04-14

1.2 背景与问题

VibePop 采用竖屏 Feed 流作为核心内容分发形式，用户通过上下滑动切换不同的互动内容（小游戏、互动相册、微生成器等）。由于内容本身包含可交互的网页元素（滚动、拖拽等），传统短视频平台的"全屏滑动切换"机制会导致内容内部手势被拦截。

核心矛盾：Feed 层需要垄断纵向滑动手势，但内容层也想用同一个手势。

1.3 需求目标

建立一套手势管理机制，在**保证 Feed 流切换体验流畅**的前提下，**允许内容内部进行复杂的交互操作**。

1.4 范围

- Feed 浏览模式下的手势处理
- 内容区域与系统手势的优先级判定
- 状态切换的视觉反馈

1.5 功能需求

1.5.1 分区手势设计

界面分为两个明确的手势区域：

区域	位置	高度	手势	功能
内容区	屏幕上半部分	屏幕高度 - 130px	点击、拖动、滑动	操作/玩内容
底部切换区	屏幕底部	80px	上下滑动	切换内容

底部切换区设计：

- 固定在屏幕底部，高度 80px
- 背景使用半透明黑色遮罩（rgba(0,0,0,0.6)），确保文字可见
- 居中显示文字提示：**"↑↓ 滑动切换内容"**
- 文字颜色：白色，字号 14px

代码块

```
┌──────────────────────────────────────────┐
│                                          │
│            内容区（游戏/互动内容）        │
│            点击、拖动、滑动操作           │
│                                          │
│                                          │
├──────────────────────────────────────────┤
│                                          │
│         ↑↓ 滑动切换内容                  │  ← 底部切换区（80px）
│                                          │
└──────────────────────────────────────────┘

```

手势规则：

- 内容区的所有手势（点击、拖动、滚动）都透传给内容本身
- 只有在底部切换区的上下滑动才会触发内容切换
- 无需状态切换，用户一眼即懂

1.5.2 切换动画

- 上下滑动底部区域时，内容卡片跟随手指移动
- 释放时根据速度/位移判断是否切换或回弹

- 切换动画时长 300ms，使用 ease-out 曲线

1.6 验收标准

1.6.1 功能验收

验收项	验收标准
分区显示	内容区占据屏幕上半部分，底部固定 80px 切换区域，文字提示清晰可见
内容区操作	在内容区内点击、拖动、滑动均正常操作内容，不触发切换
底部区切换	在底部切换区上下滑动，内容卡片跟随移动，释放后正确切换或回弹
防误触	内容区滑动到底部时，继续滑动显示回弹效果，不自动切换内容
视觉提示	底部区域文字提示 "↑↓ 滑动切换内容" 始终可见

1.6.2 性能验收

- 滑动切换帧率 ≥ 60fps
- 连续滑动 10 次无卡顿
- 底部区域响应延迟 ≤ 50ms

优化项 2：编辑器代码预览与手动编辑

2.1 基本信息

字段	内容
优化名称	编辑器代码预览与手动编辑
优先级	P1 - MVP 增强体验
状态	待评审
提出日期	2026-04-14

2.2 背景与问题

当前创作流程中，用户通过 AI 对话生成内容后，只能在预览区查看效果并通过继续对话迭代。对于具备一定技术能力的用户，直接修改代码比反复与 AI 对话更高效；对于调试场景，查看源代码也便于快速定位问题。

2.3 需求目标

在创作预览页面增加代码查看和手动编辑能力，支持：

- 切换查看 HTML/CSS/JS 源代码
- 实时编辑并同步预览效果
- 基础的代码合法性校验

2.4 功能需求

2.4.1 Tab 切换机制

在创作页面的预览区域顶部增加 Tab 切换栏：

代码块

```
┌──────────────────────────────────────┐
│  [预览]  │  [代码]                   │
├──────────────────────────────────────┤
│                                      │
│     [内容预览区 / 代码编辑器]        │
│                                      │
└──────────────────────────────────────┘

```

Tab 定义：

- **预览 Tab**：显示内容实际运行效果，与当前实现一致
- **代码 Tab**：显示源代码编辑器，支持语法高亮和编辑

切换规则：

- 默认选中「预览」Tab
- 点击 Tab 即时切换，无加载延迟
- 切换时保留当前编辑状态，不丢失未保存的修改

2.4.2 代码编辑器功能

编辑器基础能力：

- 语法高亮：HTML、CSS、JavaScript 分别着色
- 行号显示
- 代码折叠：支持按标签/函数折叠
- 自动缩进

编辑与同步：

- 在代码 Tab 修改代码后，点击「应用」按钮同步到预览 Tab
- 提供「重置」按钮恢复 AI 原始生成版本
- 修改后未保存时，Tab 上显示「●」指示器

2.4.3 代码校验与限制

保存前校验：

- HTML 结构完整性检查（标签闭合）
- 代码体积限制：≤ 50KB

错误提示：

- 语法错误时，行号旁显示红色错误标记
- 底部状态栏显示错误数量和类型

2.5 技术需求

2.5.1 编辑器选型

方案	优点	缺点	建议
Monaco Editor	功能强大，VS Code 同款	体积大 (~1MB)	推荐，功能完整
CodeMirror 6	轻量，可定制	需自行配置高亮	备选，体积敏感时
ACE Editor	成熟稳定	维护频率降低	不推荐

推荐采用 Monaco Editor，通过 CDN 按需加载，首屏不阻塞。

2.5.2 代码同步机制

- 代码 Tab 与预览 Tab 共享同一个代码状态对象
- 代码修改后，用户点击「应用」时：
  a. 执行代码校验

b. 校验通过后更新状态对象

c. 预览区 iframe 重新加载（或热更新，视技术可行性）

- 提供防抖机制：快速输入时不触发实时预览（避免闪烁）

2.5.3 安全沙箱

手动编辑的代码必须遵循与 AI 生成代码相同的安全策略：

- 在 iframe 沙箱中运行
- 禁止 `eval()`、`new Function()`
- 禁止访问 `localStorage`、`document.cookie`
- 禁止发起外部 HTTP 请求（只能加载白名单资源）

2.6 设计需求

2.6.1 Tab 视觉样式

代码块

```
┌──────────────────────────────────────┐
│  预览        代码 ●              │
│ ──────────   ──────              │
├──────────────────────────────────────┤

```

- 当前激活 Tab：底部 2px 主色指示线
- 未保存标记：Tab 文字右侧显示「●」红色圆点
- 悬浮效果：非激活 Tab 悬浮时背景轻微高亮

2.6.2 代码编辑器区域

代码块

```
┌──────────────────────────────────────┐
│  [预览]  │  [代码 ●]              │
├──────────────────────────────────────┤
│  1  │ <!DOCTYPE html>            │
│  2  │ <html>                     │
│  3  │   <head>                   │
│  4  │     <style>                │
│ ... │ ...                        │
├──────────────────────────────────────┤
```

```
│  行 42，列 15  │  2 个错误        │
│  [应用修改]  [重置]              │
└──────────────────────────────────┘

```

- 底部状态栏：显示光标位置、错误数量
- 操作按钮栏：「应用修改」（主色）、「重置」（灰色）

2.7 验收标准

2.7.1 功能验收

验收项	验收标准
Tab 切换	点击「代码」Tab，100ms 内切换到代码编辑器
语法高亮	HTML 标签、CSS 属性、JS 关键字分别显示不同颜色
代码编辑	可正常输入、删除、复制粘贴代码
应用修改	点击「应用」后，切换到预览 Tab 可见修改效果
重置功能	点击「重置」后，代码恢复为 AI 原始生成版本
错误提示	HTML 标签未闭合时，行号旁显示红色错误标记
体积限制	粘贴超过 50KB 代码时，底部提示「代码体积超出限制」
未保存提示	修改代码后未应用，Tab 上显示红色「●」标记

2.7.2 兼容性验收

- 代码编辑器在 Chrome、Safari、Edge 最新版本正常显示
- 移动端编辑器支持基础编辑（语法高亮可降级）

优化项 3：外部 AI 工具创作支持

3.1 基本信息

字段	内容
优化名称	外部 AI 工具创作支持
优先级	P2 - MVP 可选功能
状态	待评审
提出日期	2026-04-14

3.2 背景与问题

- **成本考虑**：VibePop 内置 AI 生成需承担 token 费用，随着用户增长成本上升
- **用户习惯**：部分用户已习惯使用特定外部 AI 工具（ChatGPT、Claude、Gemini 等），可能已付费订阅
- **灵活性需求**：高级用户希望使用自己偏好的模型或自定义参数进行创作

3.3 需求目标

在创作页面提供「外部 AI 创作」选项，允许用户：

- 获取完整的系统提示词 + 用户提示词
- 在外部 AI 工具中生成代码
- 将生成的代码复制回 VibePop 直接预览和发布

3.4 功能需求

3.4.1 创作入口选择

在创作页面初始状态，提供两个并列选项：

代码块

```
┌──────────────────────────────────────┐
│  选择创作方式                        │
├──────────────────────────────────────┤
│                                      │
│  [🤖 VibePop AI]    [🔗 外部AI]      │
│                                      │
│  使用平台内置AI     使用ChatGPT/     │
│  快速生成            Claude等        │
│                                      │
```

```
└──────────────────────────────────────┘

```

默认选中：「VibePop AI」（保证核心体验）

3.4.2 外部 AI 创作流程

Step 1: 输入提示词

代码块

```
┌──────────────────────────────────────┐
│  [🤖 VibePop AI]   [🔗 外部AI]  │
├──────────────────────────────────────┤
│                                      │
│  描述你想创造什么...              │
│  [_________________________]     │
│                                      │
│  [生成提示词并复制]              │
│                                      │
└──────────────────────────────────────┘

```

Step 2: 提示词卡片点击「生成提示词并复制」后，展开提示词卡片：

代码块

```
┌──────────────────────────────────────┐
│  📋 复制以下提示词到外部AI工具    │
├──────────────────────────────────────┤
│                                      │
│   ┌──────────────────────────────┐   │
│   │ 系统提示词:              │   │
│   │ {system_prompt}          │   │
│   │                          │   │
│   │ 用户提示词:              │   │
│   │ {user_prompt}            │   │
│   └──────────────────────────────┘   │
│                                      │
│  [📋 一键复制全部]               │
│                                      │
│  支持: ChatGPT · Claude · Gemini │
│                                      │
└──────────────────────────────────────┘

```

提示词组成：

- **系统提示词**：固定的格式规范、安全限制说明、HTML模板框架
- **用户提示词**：用户在输入框填写的描述

Step 3: 粘贴外部生成结果

代码块

```
┌──────────────────────────────────────┐
│  📥 粘贴外部AI生成的代码          │
├──────────────────────────────────────┤
│                                      │
│  ┌──────────────────────────────┐    │
│  │ <html>                       │    │
│  │   ...粘贴代码...             │    │
│  │ </html>                      │    │
│  └──────────────────────────────┘    │
│                                      │
│  [进入预览 →]                        │
│                                      │
└──────────────────────────────────────┘

```

Step 4: 预览

- 粘贴代码后直接跳转预览页面
- 预览区正常运行 iframe 沙箱（运行时自然过滤危险代码）

3.4.3 与现有流程的衔接

- 粘贴代码进入预览后，后续流程与 VibePop AI 生成内容一致：
  - 可继续 AI 对话修改（如果用户切回内置 AI）
  - 可手动编辑代码（见优化项 2）
  - 可发布到平台

3.5 技术需求

3.5.1 提示词模板管理

- 系统提示词以模板形式存储，包含变量占位符：

3.5.2 代码输入处理

- 支持粘贴完整 HTML 文件

- 自动提取 `<body>` 内容（如果用户粘贴了完整 HTML）
- 自动包裹必要的 HTML 骨架（如果用户只粘贴了片段）
- 代码体积前端提示（超过 50KB 显示警告，但不阻止）

3.5.3 沙箱运行

- 外部粘贴的代码与 AI 生成代码执行相同的安全沙箱策略
- 运行时违规（如尝试访问 localStorage）被静默阻止，不影响预览

3.6 验收标准

验收项	验收标准
入口选择	创作页面显示「VibePop AI」和「外部 AI」两个选项
提示词生成	输入描述后点击按钮，正确显示系统提示词 + 用户提示词
一键复制	点击复制按钮，提示词成功复制到剪贴板
代码粘贴	可粘贴 HTML 代码，字符计数实时更新
进入预览	粘贴代码后点击按钮，成功跳转到预览页面
沙箱运行	外部代码在预览区正常渲染，违规代码被静默阻止
流程衔接	外部创作的代码可正常进入发布流程

优化项 4：资源管理 Tab

4.1 基本信息

字段	内容
优化名称	资源管理 Tab

优先级	P1 - MVP 增强体验
状态	待评审
提出日期	2026-04-15

4.2 背景与问题

- 当前内容预览页面只有「预览」和「代码」两个 Tab，用户无法管理内容所需的本地资源
- 创作内容（游戏、相册、微生成器）经常需要上传图片、音频、视频等资源文件
- 目前用户只能通过外部 URL 引用资源，无法直接上传管理

4.3 需求目标

在创作预览页面增加「资源」Tab，支持：

- 上传本地资源文件（图片、音频、视频等）
- 查看已上传资源列表
- 获取资源引用路径（供代码中使用）
- 删除不再需要的资源

4.4 功能需求

4.4.1 Tab 结构扩展

将现有 Tab 从 2 个扩展为 3 个：

代码块

```
┌──────────────────────────────────────────────────┐
│  [预览]  │  [代码]  │  [资源]                    │
│          │          │  ──────                    │
├──────────────────────────────────────────────────┤
│                                                  │
│            资源管理区域                          │
│                                                  │
└──────────────────────────────────────────────────┘

```

4.4.2 资源上传

上传入口：

- 拖拽上传区域：大号虚线框，提示「拖拽文件到此处或点击上传」

- 点击上传按钮：唤起系统文件选择器

支持文件类型：

类型	格式	单文件上限	建议用途
图片	jpg, png, gif, webp, svg	5MB	游戏素材、相册图片
音频	mp3, wav, ogg	10MB	背景音乐、音效
视频	mp4, webm	50MB	视频背景、动画素材
其他	json, txt, csv	1MB	配置文件、数据

上传交互：

- 支持多文件同时上传
- 显示上传进度条
- 上传完成自动刷新资源列表
- 同名文件自动重命名（如 `bgm.mp3` → `bgm_1.mp3`）

4.4.3 资源列表

列表展示：

- 网格视图：缩略图 + 文件名 + 文件大小
- 列表视图：文件名 + 类型图标 + 大小 + 操作按钮

资源卡片：

代码块

```
┌──────────────────────┐
│    [缩略图]          │   ← 图片/视频预览，音频显示波形图标
│                      │
├──────────────────────┤
│ bgm.mp3              │   ← 文件名
│ 2.3 MB               │   ← 文件大小
├──────────────────────┤
│ [复制] [删除]        │   ← 操作按钮
└──────────────────────┘

```

操作功能：

- **复制路径**：复制资源的相对路径（如 `./assets/bgm.mp3`），供代码引用

- **删除**：删除资源，二次确认弹窗
- **预览**：点击缩略图全屏预览

4.4.4 资源引用

代码中使用：

- 资源上传后自动放入 `./assets/` 目录
- 代码中通过相对路径引用：`./assets/filename.ext`
- 提供「复制引用」按钮，一键复制 `<img src="./assets/xxx.png">` 等代码片段

4.5 技术需求

4.5.1 存储方案

方案	优点	缺点	建议
内存临时存储	简单，无需后端	刷新丢失	开发阶段
IndexedDB	本地持久化	浏览器限制容量	MVP 推荐
后端对象存储	永久保存，可分享	需要后端开发	V1.1

MVP 采用 IndexedDB：

- 资源随项目保存到本地数据库
- 发布时再打包上传到服务器

4.5.2 文件处理

- 前端校验文件类型和大小
- 图片生成缩略图（canvas 压缩）
- 大文件分片上传（V1.1 后端方案）

4.6 验收标准

验收项	验收标准
Tab 显示	预览/代码/资源三个 Tab 正常显示，可切换
文件上传	支持拖拽和点击上传，显示进度

类型限制	不符合类型的文件提示拒绝
大小限制	超出限制的文件提示「文件过大」
资源列表	上传后资源正确显示在列表中
复制路径	点击复制按钮，路径成功复制到剪贴板
删除资源	删除后列表刷新，资源不可再用
代码引用	资源路径在预览区正常加载